

Reactive REST APIs with WebFlux (Annotation Model)

A traditional REST API returns a value. A reactive REST API returns a **publisher** of a value. Spring WebFlux understands `Mono` and `Flux` as return types natively and subscribes to them automatically when a request comes in.

- `Mono<T>` for single resource responses (GET by id, POST, PUT, DELETE)
- `Flux<T>` for collection responses (GET all, streaming)
- The HTTP connection stays non-blocking end to end, from request parsing to response writing



You do not call `.subscribe()` yourself in a controller. Spring WebFlux is the subscriber.

Project Setup

Dependencies needed in `pom.xml` :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

This pulls in Netty as the default server, Project Reactor, and all WebFlux auto-configuration. Do not add `spring-boot-starter-web` alongside it.

```
# application.yml
server:
  port: 8080
  netty:
    connection-timeout: 20s # Limits the duration allowed to
    establish the initial TCP socket connection.
```

```
idle-timeout: 60s # Closes the connection if no read or write operations occur for 60 seconds.
```

Annotation-Based Controllers

WebFlux supports the same annotations as Spring MVC. For teams migrating from MVC, this is the lowest friction path.

```
@RestController
@RequestMapping("/api/v1/users")
@RequiredArgsConstructor
public class UserController {

    private final UserService userService;

    @GetMapping
    public Flux<UserResponse> getAll() {
        return userService.findAll();
    }

    @GetMapping("/{id}")
    public Mono<ResponseEntity<UserResponse>> getById(@PathVariable Long id) {
        return userService.findById(id)
            .map(ResponseEntity::ok)
            .defaultIfEmpty(ResponseEntity.notFound().build());
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Mono<UserResponse> create(@RequestBody @Valid Mono<UserRequest> req) {
        return req.flatMap(userService::create);
    }
}
```

```

@PutMapping("/{id}")
public Mono<UserResponse> update(@PathVariable Long id,
                                @RequestBody @Valid Mono
<UserRequest> req){
    return req.flatMap(request -> userService.update(id,
request));
}

@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public Mono<Void> delete(@PathVariable Long id) {
    return userService.delete(id);
}
}

```



Pass `Mono<UserRequest>` as the request body instead of `UserRequest` directly. This keeps the deserialisation non-blocking and lets you chain operators on the incoming body before it is fully read.

ResponseEntity vs Direct Return

Both work. Use `ResponseEntity` when you need to control headers or status codes dynamically. Use direct return when the status is always the same.

```

// direct return -- simpler, status set via @ResponseStatus
@GetMapping
public Flux<UserResponse> getAll() {
    return userService.findAll();
}

// ResponseEntity -- when status depends on result
@GetMapping("/{id}")
public Mono<ResponseEntity<UserResponse>> getById(@PathVariable Long id) {
    return userService.findById(id)
}

```

```

        .map(ResponseEntity::ok)
        .defaultIfEmpty(ResponseEntity.notFound().build
    ));
}

```

SSE Streaming Endpoint

Server-Sent Events let the connection stay open and push data as it becomes available.

```

@GetMapping(value = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<UserResponse> streamUsers() {
    return userService.findAll();
}

```

SSE in Spring MVC (with Virtual Threads)

Spring MVC supports SSE through `SseEmitter`. Unlike WebFlux, the connection holds a thread for its entire duration, which is exactly why virtual threads matter here. A platform thread held open for an SSE stream is wasteful, a virtual thread held open is cheap.

```

@GetMapping("/stream/users")
public SseEmitter streamUsers() {
    SseEmitter emitter = new SseEmitter(Long.MAX_VALUE);

    Executors.newVirtualThreadPerTaskExecutor().execute(() -> {
        try {
            userRepository.findAll().forEach(user -> { // blocking JDBC, fine on VT
                emitter.send(user);
            });
            emitter.complete();
        } catch (Exception e) {

```

```

        emitter.completeWithError(e);
    }
});
return emitter;
}

```



At moderate scale, a few thousand concurrent SSE connections, `SseEmitter` with virtual threads performs well and is far simpler to write and debug. At extreme scale, hundreds of thousands of concurrent connections, WebFlux's back-pressure and lower per-connection overhead start to matter.

Exception Handling

`@RestControllerAdvice` works in Spring WebFlux (annotation model) very similarly to Spring MVC, and it *is* the standard way to do global exception handling there too.

In WebFlux, exceptions may happen **later in the reactive pipeline**, so returning a reactive type is natural:

```

@GetMapping("/{id}")
public Mono<UserResponse> get(@PathVariable Long id) {
    return userService.findById(id)
        .switchIfEmpty(Mono.error(new UserNotFoundException(id)));
}

```

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public Mono<ResponseEntity<ErrorResponse>> handle(UserNotFoundException ex) {
        return Mono.just(ResponseEntity

```

```
        .status(HttpStatus.NOT_FOUND)
        .body(new ErrorResponse(ex.getMessage())));
    }
}
```

Returning non-reactive (`ResponseEntity<...>`) can also work, but reactive return types integrate cleanly with the runtime.